



香港中文大學 (深圳)
The Chinese University of Hong Kong

CSC 5003 Algorithm Art and Programming Practice Lecture 9: Dynamic Programming 2

Jingbang Chen
School of Data Science (SDS)
The Chinese University of Hong Kong, Shenzhen



Tallest Billboard

- ▶ You are given an array `rods`, where `rods[i]` is the length of the i -th rod.
- ▶ You want to build a billboard with two steel supports of **equal height**.
To do this, you may partition the rods into three groups:
 - ▶ rods used in the left support,
 - ▶ rods used in the right support,
 - ▶ rods not used.
- ▶ Both supports must have the same total height.
- ▶ Return the **maximum possible height** of each support.
If it is impossible to make two supports of equal height, return 0.



Solution

- ▶ A direct DP on the heights of both supports would be awkward, because we would need to track two values:
 - height of the left support
 - height of the right support
- ▶ Instead, the key trick is to track only their **difference**.
- ▶ $F[d]$ denotes the difference between the two supports is d , the maximum possible height of the **shorter** support is $dp[d]$.



Solution

- ▶ For each rod x , from every existing state $F[d]=h$, we have three choices:
 - The state remains unchanged:
 - $\text{new_dp}[d] = \max(\text{new_dp}[d], h)$
 - Add the rod to the taller support
 - $\text{new_dp}[d + x] = \max(\text{new_dp}[d + x], h)$
 - Add the rod to the shorter support
 - $\text{new_dp}[\text{abs}(d - x)] = \max(\text{new_dp}[\text{abs}(d - x)], h + \min(d, x))$
- (Why this is correct?)



Number of Ways to Wear Different Hats to Each Other

- ▶ There are n people, numbered from 0 to $n - 1$.
- ▶ Each person has a list of hats they are willing to wear. The input is given as: `hats[i]`,
- ▶ where `hats[i]` is the list of hat numbers that person i likes.
- ▶ You want to assign **one hat to each person** such that:
- ▶ each person gets exactly one hat they like;
- ▶ no two people wear the same hat.
- ▶ Return the number of different valid assignments, modulo:
 - ▶ $10^9 + 7$



Solution

- ▶ **bitmask DP:** process hats one by one, and decide whether to assign the current hat to one of the people who like it.
- ▶ This works because the number of people is small, so we can use a bitmask to record which people have already received hats.



Solution

- ▶ Let $dp[h][mask]$ denote the number of ways to assign hats from 1 to h such that the set of people who already have hats is exactly $mask$.
- ▶ Do not use hat h
 - Then the set of assigned people stays the same:
 - $dp[h][mask] += dp[h - 1][mask]$
- ▶ Give hat h to one person who likes it
 - $dp[h][mask] += dp[h - 1][prevMask]$ for every valid person p in $hatToPeople[h]$, $prevMask = mask$ without person p



Solution

- ▶ Since $dp[h]$ depends only on $dp[h-1]$, we can compress the first dimension.
- ▶ Why We Process Hats Instead of People?
- ▶ Can we write it in a Top-Down Memoization?
 - $dfs(hat, mask)$ = number of ways starting from hat number `hat` when `mask` indicates which people are already assigned



Profitable Schemes

- ▶ There are n members in a gang.
- ▶ You are given two arrays:
 - $\text{group}[i]$: the number of members required to commit the i -th crime
 - $\text{profit}[i]$: the profit earned from the i -th crime
- ▶ A **scheme** is any subset of crimes such that:
 - the total number of gang members used is at most n
 - the total profit is at least minProfit
 - Each crime can be chosen at most once.
- ▶ Return the number of different schemes, modulo $10^9 + 7$



Solution

- ▶ **Counting knapsack DP.**
- ▶ Each crime can be: selected once, or not selected
- ▶ So it looks like a 0/1 knapsack.

But unlike ordinary knapsack, we are not maximizing profit. We are:

- ▶ limiting the number of members used
- ▶ counting how many subsets achieve profit at least `minProfit`

So the DP should count the number of ways, not store the best value.



Solution

- ▶ Let $dp[j][p]$ denote the number of schemes that use exactly j gang members and achieve profit **at least** p .
- ▶ Notice that we **cap profit at minProfit**:
- ▶ If a scheme has profit larger than $minProfit$, we do not need the exact value anymore. So whenever profit exceeds $minProfit$, we clamp it back to $minProfit$.
- ▶ $dp[j][newProfit] += dp[j - g][oldProfit]$



Longest Path With Different Adjacent Characters

- ▶ You are given a tree with n nodes numbered from 0 to $n - 1$.
- ▶ You are also given a string s of length n , where $s[i]$ is the character assigned to node i .
- ▶ Return the length of the **longest path** such that **any two adjacent nodes on the path have different characters**.



Solution

- ▶ **Tree DP:** For each node u , we want to know the longest valid downward chain starting from u .
- ▶ Once we know the best downward chains from the children, we can try to combine two of them through u to form a longer path.



Solution

- ▶ Suppose we are processing node u .
- ▶ For each child v of u , we recursively compute:
- ▶ $childLen = dfs(v)$
- ▶ If:
- ▶ $s[v] \neq s[u]$
- ▶ then the path from v can be extended upward through u , so it becomes a candidate chain of length:
- ▶ $childLen + 1$
- ▶ Among all such valid children, we want the **two largest** downward chains:
- ▶ $best1$: the largest valid child contribution
- ▶ $best2$: the second largest valid child contribution
- ▶ Why two?
- ▶ Because the longest path passing through u may use:
- ▶ one chain from one child,
- ▶ another chain from another child,
- ▶ plus node u in the middle.
- ▶ So the path length through u would be:
- ▶ $best1 + best2 + 1$
- ▶ where $best1$ and $best2$ are measured as lengths below u excluding u itself.
- ▶ Equivalently, if we store child contributions already as full chain lengths starting from the child, then:
- ▶ $pathThroughU = 1 + best1 + best2$



Solution

- ▶ Suppose we are processing node u .
- ▶ For each child v of u , we recursively compute:
- ▶ $\text{childLen} = \text{dfs}(v)$
- ▶ If $s[v] \neq s[u]$
- ▶ then the path from v can be extended upward through u , so it becomes a candidate chain of length:
- ▶ $\text{childLen} + 1$



Solution

- ▶ Among all such valid children, we want the **two largest** downward chains:
 - best1: the largest valid child contribution
 - best2: the second largest valid child contribution
- ▶ Because the longest path passing through u may use:
 - one chain from one child,
 - another chain from another child, plus node u in the middle.
- ▶ So the path length through u would be: $\text{best1} + \text{best2} + 1$, where best1 and best2 are measured as lengths below u excluding u itself.
- ▶ Equivalently, if we store child contributions already as full chain lengths starting from the child, then:
 - $\text{pathThroughU} = 1 + \text{best1} + \text{best2}$



Solution

- ▶ The value returned from u should be the best single downward chain starting at u , namely: $1 + \text{best1}$ because it can extend through at most one child if it must remain a simple path.
- ▶ We maintain a global variable `ans`: At every node u , after finding the two best child contributions, we update:
 - $\text{ans} = \max(\text{ans}, 1 + \text{best1} + \text{best2})$
- ▶ This considers the best path whose highest point is u .
- ▶ Since every simple path in a tree has a unique highest point (its LCA-like turning point), checking all nodes this way covers all possible paths.



Largest Color Value in a Directed Graph

- ▶ There is a directed graph with n nodes, numbered from 0 to $n - 1$. Each node has a color.
- ▶ The color value of a path is defined as the number of occurrences of the most frequent color along that path. Return the largest color value among all valid paths in the graph.
- ▶ If the graph contains a directed cycle, return -1.



Solution

- ▶ If the graph contains a cycle, then the answer is immediately -1.
- ▶ If the graph is acyclic, then we can process the nodes in **topological order**, and for each node keep track of:
 - ▶ for every color c , what is the maximum number of occurrences of color c on any path ending at this node?
- ▶ **DAG DP**: Since there are only 26 lowercase letters, we can maintain a DP array of size 26 for each node.



Solution

- ▶ Let $dp[u][c]$ denote the maximum number of occurrences of color c along any valid path that ends at node u .
- ▶ Our answer will be the maximum value among all $dp[u][c]$.
- ▶ $dp[v][c] = \max(dp[v][c], dp[u][c] + (colors[v] == c))$



Remove Boxes

- ▶ You are given several boxes arranged in a row. Each box has a color represented by a positive integer in the array `boxes`.
- ▶ You may repeatedly perform the following operation:
- ▶ choose a contiguous group of k boxes with the same color, remove them, gain $k * k$ points.
- ▶ After removing those boxes, the remaining boxes close up and become adjacent again.
- ▶ Return the maximum points you can obtain by removing all the boxes.



Why This Problem Is Hard

- ▶ A straightforward interval DP like $f[l][r]$ is not enough.
- ▶ The reason is that the best way to remove boxes inside an interval may depend on whether there are **extra boxes of the same color outside the interval** that can later be merged with one endpoint.
- ▶ For example, sometimes it is better **not** to remove a box immediately, because after removing the middle part it can join with another box of the same color and produce a larger square reward.
- ▶ So the state must remember some information from outside the interval.



Solution

- ▶ $F[l][r][k]$ denote the maximum points obtainable from the subarray $\text{boxes}[l..r]$, assuming there are already k extra boxes equal to $\text{boxes}[r]$ attached to the right of r .
- ▶ In other words:
- ▶ we are solving interval $[l, r]$,
- ▶ but box $\text{boxes}[r]$ can be grouped together with k same-colored boxes that have been "saved" from outside.
- ▶ This extra parameter is the key to restoring optimal substructure.



Why This State Works

- ▶ Suppose `boxes[r]` has color c .
- ▶ If there are k extra boxes of color c attached to it, then if we remove `boxes[r]` together with them, the reward is: $(k + 1)^2$
- ▶ if `boxes[r]` stands alone inside the interval,
- ▶ or even larger if we first merge it with matching boxes inside $[l, r-1]$.
- ▶ So k tells us how valuable it is to keep `boxes[r]` for later merging.



Transition

- ▶ **Option 1: Remove boxes[r] now**
- ▶ First solve $[l, r-1]$, then remove boxes[r] together with the k extra boxes:
 - $dp[l][r][k] = dp[l][r-1][0] + (k+1)^2$
- ▶ This corresponds to treating boxes[r] as the last group removed.
- ▶ **Option 2: Merge boxes[r] with some earlier box of the same color**
- ▶ If there exists some m with: $l \leq m < r$ and $boxes[m] == boxes[r]$
- ▶ then instead of removing boxes[r] separately, we may first remove all boxes in (m, r) , so that boxes[m] becomes adjacent to boxes[r].
- ▶ Then boxes[r] can be merged into the group containing boxes[m].
- ▶ So we try: $dp[l][r][k] = \max(dp[l][m][k+1] + dp[m+1][r-1][0])$
- ▶ for every m such that $boxes[m] == boxes[r]$.



Transition

- ▶ $dp[m+1][r-1][0]$: clear the middle part so $boxes[m]$ and $boxes[r]$ can touch;
- ▶ $dp[l][m][k+1]$: now $boxes[m]$ has one more same-colored box attached to it, namely $boxes[r]$, plus the original k extras.
- ▶ Let's try implementing with memorization search!